Creative Software Design
5. Lab2
Make & CMD Arguments

Dept. of Computer Science

Hanyang University

[IMPORTANT] Submission Guide

- Pull to your project directory before each lab session
  - At your VM instance

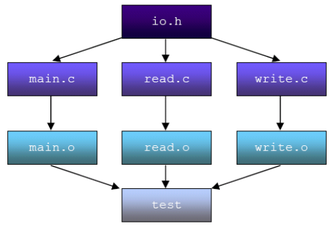
```
$ cd Projects/{your directory}
$ git pull origin main
$ cd week05_2
```

- Make sure that your file/directory names are correct
  - Each lab files MUST be placed in ***week05_2***
  - Each lab file names MUST be ***example0x.cpp***
  - You don't have to be creative at this !
- Make sure that your source files are correct
  - No compile errors or warnings
  - Correct output should be appeared in execution (without errors)
  - Any of incorrect files loses all points

How to Build Executable Files

Manually build

```
g++ -o test read.cpp write.cpp main.cpp  # compile and link
# or
g++ -c read.cpp write.cpp main.cpp # compile
g++ -o test read.o write.o main.o  # link
```



Makefile

```
test : read.o write.o main.o
    g++ -o test read.o write.o main.o

main.o : io.h main.cpp
    g++ -c main.cpp

read.o : io.h read.cpp
    g++ -c read.cpp

write.o: io.h write.cpp
    g++ -c write.cpp
```

Dependency information

#1 – Makefile: Targets

- Write `Makefile` for two targets
  - Reuse the example03.cpp from the previous session

```
$ cp ../week05_1/example03.cpp ./example01.cpp
$ cp ../week05_1/add.* ./
$ cp ../week05_1/add_dgb.cpp ./
```

  - `example01` with `example01.o` and `add.o`
  - `example01_dbg` with `example01.o` and `add_dbg.o`

```
// example01.cpp
#include <iostream>
#include "add.h"

int main(void) {
  std::cout << add(1, 2)
    << std::endl;
  return 0;
}
```

```
// add.h
int add(int x, int y);
```

```
// add.cpp
int add(int x, int y) {
  return x + y;
}
```

```
// add_dbg.cpp
#include <iostream>

using namespace std;

int add(int x, int y) {
  cout << "calling add(" << x
    << ", " << y << ")" << endl;
  return x + y;
}
```

#2 – Makefile: Dependencies

- Complete given source files and Makefile

```
// example02.cpp
#include <iostream>
#include "point_print.h"

int main(void) {
  Point p[] = {{8, 3}, {7, 5}, {5, 1},
        {4, 9}};
  print_center_of_points(p, sizeof(p)
        / sizeof(p[0]));
  return 0;
}
```

```
// point.h
#ifndef __POINT_H__
#define __POINT_H__
typedef struct point {
  int x;
  int y;
} Point;
#endif //__POINT_H__
```

```
// point_sum.h
#ifndef __POINT_SUM_H__
#define __POINT_SUM_H__
#include "point.h"
void point_sum(Point*, int, Point&);
#endif //__POINT_SUM_H__
```

```
// point_print.h
#ifndef __POINT_PRINT_H__
#define __POINT_PRINT_H__
#include "point.h"
void point_print(Point*, int);
#endif //__POINT_PRINT_H__
```


#3 – CMake

- Complete CMakeLists.txt

```
$ mkdir build; cd build; cmake ../
```

```
// example02.cpp
#include <iostream>
#include "point_print.h"

int main(void) {
  Point p[] = {{8, 3}, {7, 5}, {5, 1},
        {4, 9}};
  print_center_of_points(p, sizeof(p)
        / sizeof(p[0]));
  return 0;
}
```

```
// point.h
#ifndef __POINT_H__
#define __POINT_H__
typedef struct point {
  int x;
  int y;
} Point;
#endif //__POINT_H__
```

```
// point_sum.h
#ifndef __POINT_SUM_H__
#define __POINT_SUM_H__
#include "point.h"
void point_sum(Point*, int, Point&);
#endif //__POINT_SUM_H__
```

```
// point_print.h
#ifndef __POINT_PRINT_H__
#define __POINT_PRINT_H__
#include "point.h"
void point_print(Point*, int);
#endif //__POINT_PRINT_H__
```

Command-line Arguments

- A simple program to print all command-line arguments

```
#include <stdio.h>

int main(int argc, const char **argv) {
  for (int i = 0; i < argc; ++i) {
    printf("%s\n", argv[i]);
  }
  return 0;
}
```

- You may need ***string-to-number*** conversion
  - **`stdlib.h: atoi(),atol(),atoll(),atof()`**

    @see Online API documents or `$ man atoi`

```
#include <stdio.h>
#include <stdlib.h> // or #include <cstdlib>

int main(int argc, const char **argv) {
  for (int i = 1; i < argc; ++i) {
    printf("%d\n", atoi(argv[i]));
  }
  return 0;
}
```


#4 – Data Masking

- Partially hide the specified section of a code using '*'
- All required input will be passed from command line
  - A number or code, a starting position, and an ending position

```
wsul@splab:~/Projects/ITE1015/2025_ite1015_2025012345/week05_2$ ./example04 12345678 0 2
**345678
wsul@splab:~/Projects/ITE1015/2025_ite1015_2025012345/week05_2$ ./example04 12345678 1 3
1**45678
wsul@splab:~/Projects/ITE1015/2025_ite1015_2025012345/week05_2$ ./example04 12345678 4 5
1234*678
wsul@splab:~/Projects/ITE1015/2025_ite1015_2025012345/week05_2$ ./example04 12345678 7 0
12345678
```

#5 – Inputstream vs. FileInputStream

- Based on "example02.cpp" in week03_2

```
$ cp ../week03_2/sample.txt ./
$ cat sample.txt
$ ./example05 sample.txt (should be same as cat)
```

```
#include <iostream>
#include <fstream>

using namespace std;

int main(int argc, char** argv) {
   ...
   fstream fin;
   fin.open(....);
   while (getline(fin, line)) {
     ...
   }
   ...
   fin.close();
   return 0;
}
```


All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week05_2")

```
# You are at {Your local project directory}/week05_2
$ git add Makefile *.cpp *.h
$ git commit –m "submission of week05_2"
$ git push origin main
```